



UNIVERSIDAD · ECCI

Predicción de fuga de clientes

Integrantes

Laura Alejandra Becerra Feo
Sergio Steban Aguirre Alfonso

Docente

Fredy Alexander Orjuela López

Universidad ECCI
Ingeniería industrial
Sistemas de información gerencial

1. Definición de métricas en el contexto de churn

Las siguientes métricas fueron obtenidas del experimento de Machine Learning y se interpretan desde la óptica de la toma de decisiones gerenciales en una empresa de telecomunicaciones:

Métrica	Definición	Implicación para el negocio
Accuracy	Porcentaje de predicciones correctas sobre el total de casos evaluados.	Un modelo con Accuracy = 0.76 acertó en el 76% de los clientes. Advertencia: un modelo que prediga 'nadie se va' obtendría 73% sin ser útil, pues esa es la proporción base de clientes que no se fugan.
Recall	De todos los clientes que realmente se fugan, ¿cuántos detecta el modelo?	Recall = 0.35 significa que se detectan 35 de cada 100 desertores reales. Es la métrica crítica para no dejar escapar clientes valiosos.
Precision	De todos los clientes marcados como 'fugará', ¿cuántos realmente lo harán?	Precision = 1.00 en CatBoost significa que cada alarma es verdadera, pero el modelo lanza muy pocas (Recall = 0.11), dejando escapar al 89% de los desertores.
F1-Score	Media armónica entre Recall y Precision. Penaliza los extremos desequilibrados.	Un modelo con Precision = 1.00 y Recall = 0.11 obtiene F1 = 0.20, evidenciando que sacrifica demasiado la detección de desertores.
AUC	Área bajo la curva ROC. Mide la capacidad discriminativa del modelo en todos los umbrales posibles.	AUC = 0.50 equivale a adivinar al azar; 1.00 es perfecto. En esta tabla los valores rondan 0.55–0.57, lo que indica capacidad discriminativa modesta con el conjunto de datos sintético.
Kappa / MCC	Corrigen el efecto del desbalance de clases en la evaluación del modelo.	Un modelo que siempre predice 'no fuga' obtiene Accuracy = 0.73 pero Kappa = 0 y MCC = 0, demostrando que no aprendió nada real. Son más confiables que el Accuracy cuando las clases están desbalanceadas.

2. Análisis de costos por tipo de error

La campaña de retención consiste en ofrecer un mes gratis de servicio a cada cliente identificado como 'en riesgo de fuga'. Esto obliga a evaluar dos tipos de error con impactos económicos distintos:

Tipo de error	Descripción	Impacto en el negocio
Falso Positivo (FP)	El modelo predice fuga, pero el cliente se habría quedado.	Se entrega un mes gratis innecesariamente. Costo puntual, recuperable en el siguiente ciclo de facturación.
Falso Negativo (FN)	El modelo no detecta al desertor y no se activa la campaña.	El cliente cancela. Pérdida permanente del Customer Lifetime Value (CLV). Adquirir un nuevo cliente cuesta entre 5 y 7 veces más que retener uno existente.

3. Modelo recomendado para la campaña

Modelo seleccionado: Extra Trees Classifier. Alternativa conservadora: Random Forest.

Justificación estratégica:

- CatBoost, LightGBM y Extreme Gradient Boosting tienen Precision = 1.00 pero Recall = 0.11: detectan menos del 12% de los desertores reales. No son adecuados para campañas de retención masiva.
- Decision Tree registra el mayor Recall (0.3567) pero genera un alto volumen de alertas incorrectas (Kappa = 0.09, F1 = 0.34), lo que implica un gasto elevado en meses gratis innecesarios.
- Extra Trees Classifier ofrece el mejor balance: Recall = 0.23, F1 = 0.29, Kappa = 0.12 y AUC competitivo de 0.572. Captura más desertores reales sin disparar excesivamente el costo de la campaña.
- Si la gerencia financiera establece un presupuesto limitado para la campaña, Random Forest (Recall = 0.19, Precision = 0.454) es la alternativa que reduce el desperdicio de incentivos.

Modelo	Recall	F1-Score	Veredicto
CatBoost / LightGBM / Extreme GB	0.1169	0.2094	No apto — detecta menos del 12% de desertores
Random Forest	0.1937	0.2715	Alternativa conservadora (presupuesto limitado)
Extra Trees Classifier (Recomendado)	0.2296	0.2935	Mejor balance Recall / costo de campaña
Decision Tree	0.3567	0.3438	Alto volumen de falsos positivos — costoso

Conclusión: el costo de entregar un mes gratis a un cliente que no se iba (Falso Positivo) es recuperable. La pérdida de un cliente que cancela sin intervención (Falso Negativo) es permanente. Esta asimetría justifica priorizar el Recall sobre la Precision en la selección del modelo.

4. Problemas de infraestructura a 10 millones de clientes diarios

Al escalar el sistema a análisis en tiempo real sobre 10 millones de clientes diarios en arquitectura cloud (AWS / Azure), los algoritmos de tipo ensemble presentan tres cuellos de botella críticos frente a los modelos lineales:

Cuello de botella	Algoritmos pesados (Random Forest / Ensembles)	Modelos simples (Logistic Regression / Ridge)
Memoria RAM	RF con 70 árboles sobre 200k registros requiere varios GB. Con 10M de registros en streaming, la huella supera las instancias estándar.	Modelos lineales: ecuación matricial de pocos MB. Ejecutable en instancias serverless de bajo costo.
Latencia de inferencia	Cada predicción atraviesa decenas de árboles. A 10M de eventos/día, la latencia acumulada puede superar los SLA del sistema CRM.	Inferencia = multiplicación matricial simple. Ejecutable en microsegundos como Lambda function.
Reentrenamiento	Requiere reentrenamiento completo periódico sin aprendizaje incremental nativo. Jobs batch costosos en clúster	Reentrenamiento rápido. Compatible con aprendizaje en línea (online learning) para actualización

	Spark o GPU.	incremental.
--	--------------	--------------

5. ¿Justifica un 2% adicional de Accuracy multiplicar por diez el costo de servidores?

En la mayoría de los escenarios, no. La decisión debe estar respaldada por un análisis costo-beneficio explícito: si el ingreso recuperado por la mayor tasa de detección supera el costo incremental de la infraestructura, la inversión es justificable. De lo contrario, la diferencia marginal en precisión no compensa el impacto en el presupuesto de TI.

Para telecomunicaciones con millones de clientes en tiempo real, la arquitectura recomendada desde la perspectiva de Sistemas de Información Gerencial es:

- Entrenar el modelo complejo (GBM / XGBoost) en un entorno offline con el poder computacional disponible.
- Exportar el modelo entrenado como archivo serializado ligero en formato ONNX o PMML.
- Desplegarlo en un microservicio de inferencia de baja latencia (contenedor Docker o Lambda function en AWS), separando el costo del entrenamiento del costo de la inferencia en producción.

Esta arquitectura permite capturar la calidad predictiva de los modelos ensemble sin incurrir en el costo computacional de reentrenarlos en cada ciclo de predicción en producción.